SNT Hub project integration specification

Bundle contract and recommended DIPL organization | SNT Hub v0.1.0

Ondrej Pego Jaura

10 October 2026

Contents

1 Purpose and scope 1

2 Bundle contract 1

3 Recommended DIPL model structure 2
- 3.1 Model the user's concepts first . . . 2
- 3.2 Keep build and runtime choices in one DIPL contract . . . 2
- 3.3 Emerging patterns across Hub projects . . . 3
- 3.4 Separate parameters from output decisions . . . 3
- 3.5 Define schemas before example values . . . 4
- 3.6 Compose each example in a predictable order . . . 4
- 3.7 Discover and express meaningful dependencies . . . 5
- 3.8 Keep provenance inspectable . . . 5

4 Adapter and workspace behavior 5

5 Evidence and publication checklist 6

6 Related resources 6

7 Maintaining this document 6

1 Purpose and scope

SNT Hub publishes reproducible parameter integrations for established scientific codes. A Hub project pairs a **pinned original-code revision** with a **Hub-owned DIPL layer**, reviewed example recipes, and evidence of what has been checked. The original solver retains its own input format and numerical behavior. An external adapter may prepare those native inputs first; direct SNT3 integration or generated static parameters may replace that adapter later.

This document distinguishes the **bundle contract** required by the current Hub from **recommended model organization**. The recommendations are conventions for contributors, not new DIPL syntax or claims about every code. The new-project guide describes the contribution steps; the DIPL specification defines language semantics.

The Hub release version in the PDF title comes from the repository's VERSION file. It labels this Hub publication; a project's pinned upstream source revision and the catalogue's schema version are independent identifiers.

2 Bundle contract

Every published project has a stable lowercase ID and a directory `projects/<id>/`:

```
projects/<id>/
  README.md                 scope, prerequisites, evidence, limitations
  project.json              registry record and Hub adapter capabilities
  setups.json               reviewed example IDs and capabilities
  source/                   pinned upstream Git submodule
  dipl/
    pyproject.toml          installable adapter package
    profiles/schemas/       reusable DIPL definitions
    examples/<example>/     one DIPfile and its local inputs per setup
    src/                    native-input adapter and optional build/run hooks
    tests/                  parity and workflow checks
  docs/parameters/          checked-in parsed parameter reports
```

`source/` is the code owner's pinned checkout. Hub development must not edit it to make an adapter work. `dipl/` is the integration layer and may be edited in a fetched local workspace; setup locks then record the edited state.

The project record identifies the canonical source URL and full revision, adapter package and executable, setup manifest, integration mode, maintainer support, validation level, evidence, limitations, and review date. Set `hub.output_plan` to `true` only when the adapter records and replays its resolved native mappings in DIPH5. Declare build and run capabilities only when the corresponding project hooks exist and have been exercised. The setup manifest maps each Hub example ID to its DIPL directory and exact upstream example path. A recipe is `complete` only if it prepares every required input or verified asset; otherwise mark it `native-inputs-only` and require the user's explicit `--inputs-only` choice.

The public registry derives from these records. **Integration mode**, **maintainer support**, and **validation evidence** are separate dimensions. A matching parameter file is settings parity; it is not proof of a successful solver run or scientific validation. See the status definitions.

3 Recommended DIPL model structure

3.1 Model the user's concepts first

Organize public parameter paths by the concepts a researcher chooses or inspects. Common top-level groups are `geometry`, `materials`, `physics`, `numerics`, `initial_state`, and `output`; use only groups that fit the code. Keep a stable, descriptive path for each concept. Within a group, put identity and selection fields first, then dimensions and physical values, then numerical controls, then derived or optional settings. Keep repeated entities as arrays or schema instances rather than numbered, unrelated fields.

The native file's section or command order belongs in the renderer. A DIPL tree may follow a native format where that makes the model clearer, but it should not be forced to mirror parser quirks. Record the mapping to native names in metadata or an adapter mapping so readers can move from a parameter to the generated file.

3.2 Keep build and runtime choices in one DIPL contract

When users can choose compile-time settings that change the capabilities or behavior of the built executable, represent those choices as DIPL nodes in the same project contract as the runtime settings. Use a distinct namespace such as `build.*` alongside domain groups such as `physics.*`, `numerics.*`, and `output.*`. A single evaluated DIPL project should make the selected executable configuration and case configuration inspectable and overridable together.

The adapter may emit these nodes to different targets: build options can become a compiler/environment profile or build-system arguments, while runtime settings become native case files. This difference in destination does not require a second parameter model. Record the build mappings and relevant compiler, library, and source versions in the build output or lock. Host facts that the user does not choose, secrets, and arbitrary site-specific paths belong in environment or workflow configuration unless they are explicitly modeled as supported choices.

Keep the build workflow capability separate from the parameter contract. A project may model and generate a build profile before it offers a verified Hub `build` hook; in that case, label the workflow as manual or

unsupported and do not imply that the Hub compiled the code. Avoid adding build nodes for knobs the code does not expose or the integration cannot apply.

Use DIPL tags to let the adapter discover build and runtime nodes without hard-coding a second list of paths. A project can use a consistent pair such as `<code>:config` for user-selectable build settings and `<code>:param` for runtime settings. Select them with the DIPL inspection API (for example, `env.select("?", tags_all=["openfoam:config"])`). Tags classify nodes for discovery; they do not define conversion or execution behavior. Keep native mappings in `?native` metadata or the adapter's explicit mapping layer, and validate that required tagged nodes and mappings are present.

3.3 Emerging patterns across Hub projects

Across the current integrations, reusable schemas define typed parameter groups, example profiles select and specialize those groups, and `?native` metadata connects public DIPL paths to code-level names. Descriptions, units, options, and conditions make the contract inspectable; adapters then translate the evaluated contract into each code's native files. These patterns are shared even though the projects use very different input formats.

Use code-namespaced tags to classify nodes that an adapter needs to discover. The current projects use `<code-id>:param` for mapped runtime parameters (the Quantum ESPRESSO adapter uses the short namespace `qe`); where users can select build settings, use `<code-id>:config`. Keep the tags on DIPL node declarations so schema reuse carries the classification into examples. Do not use tags as a substitute for `?native`, output targets, or serialization rules.

The current examples also show that `?native` alone does not specify the full output format. A native path may identify a field while the adapter still needs to know its file, section or card, order, conditional presence, and any required conversion. Keep those format decisions in the adapter's explicit mapping or recorded output plan, and validate that every tagged runtime node is handled. Avoid duplicating the whole parameter catalog in Python when tags and native metadata can drive discovery; retain small format-specific ordering and policy tables where the native format requires them.

Expressions and branching are most useful for short, deterministic relationships or meaningful alternative settings that users should inspect and override. The current integrations use these features selectively, while most settings remain direct typed values. Add a derived node only when it represents a meaningful setting; keep formatting, file selection, and native-only decisions in the adapter/output plan. Test representative overrides against both the recorded plan and generated native files, then use snapshot replay to check that recorded decisions reproduce the same output.

3.4 Separate parameters from output decisions

The boundary is the evaluation of the DIPL project. **DIPL nodes describe the effective simulation settings** that a researcher can inspect, validate, or override. After evaluation, the **adapter-owned output plan records how those settings become native input**. `describe_outputs(env, outputs)` resolves mappings and decisions; `plan_resolved(env, outputs, context)` renders files from that resolved plan. The output plan is saved in DIPH5 so snapshot replay can use the recorded values and decisions without reevaluating the original DIPL profiles.

Put in	When	Example
DIPL node	The value or choice has meaning to the user, affects the simulation, and should be inspectable or overridable.	A cutoff, grid dimension, or choice of sampling mode.
DIPL expression	A meaningful setting is a short, deterministic consequence of other settings and is useful to inspect in its own right.	A cell midpoint derived from cell length.
Output plan record	The information selects, names, omits, converts, or derives native output, but is not itself an independent user setting.	A native key and target; whether a command is emitted; a derived native-only switch.

Put in	When	Example
Renderer	The work is file formatting, ordering, staging, or substantial computation.	Namelist syntax, table layout, or copying a pinned input asset.

A semantic DIPL node need not correspond to one native assignment. One node may select a native card variant or affect several commands; several nodes may form one native record. Conversely, a native-only value can be recorded with `outputs.add_value(...)` without inventing a public DIPL node. Do not create nodes solely to carry export flags, native names, or intermediate formatting state. Do not hide a scientifically meaningful user choice inside adapter policy merely because it changes which native line is written.

Give each output mapping a stable ID, target and native key. Record its source path or dependencies, resolved value, active decision, and any intentional profile replacement. Render from the saved plan rather than recomputing profile choices during replay. Keep renderer details out of parameter descriptions; document non-obvious conversions and test that overrides change both the plan and the native file as expected. A saved plan guarantees replay of its mappings and decisions; any external source templates or assets still needed by the renderer must be pinned and documented.

3.5 Define schemas before example values

Put reusable type and validation definitions in `profiles/schemas/`. Group files by coherent domain, not by individual example or by arbitrary file length. A small project may need one schema file; a larger project can split units, geometry, physical models, numerical methods, and output settings. Name schemas for the concept they define. Prefer a small base schema plus explicit optional extensions over one schema with every possible field.

Within a schema, order declarations for a reader: identifying choices, required values, optional values, then derived values. Give each public field a meaningful type, unit where applicable, a concise `?descr`, and a condition or options list when there is a real constraint. Add a source URL or rationale when the meaning is code-specific or surprising. Describe the physical meaning and valid range; avoid repeating the field name as its description. Record native names when one exists. Constraints must reflect the pinned code, not a guessed universal rule.

For example, this defines a reusable concept and sets one example value:

```
$schema simulation_geometry
  box_length float
    !condition ({.} > 0)
    ?descr "Physical length of one side of the simulation box."

geometry : simulation_geometry
  box_length = 10.0
```

This is an organizational example; a real model should use the code's appropriate unit system and native mapping. Schema names and example values should remain separate so another setup can reuse the same constraint without copying it.

3.6 Compose each example in a predictable order

An example directory has one `DIPfile` as its entry point. Register reusable schema files before the example's value file. A recommended reading order is:

1. Shared units and basic types, if the code needs custom definitions.
2. Reusable domain schemas, from broad structures to optional extensions.
3. Shared profiles or defaults that are genuinely common to several examples.
4. The example's settings and data sources.
5. Any meaningful DIPL values derived from the effective settings. Declare native output mappings in the adapter after evaluation.

For instance:

```
code[]
  file = "../../profiles/schemas/geometry.dip"

code[]
  file = "../../profiles/schemas/numerics.dip"

code[]
  file = "settings.dip"
```

The actual `DIPfile` order must obey DIPL's loading rules. Register override files using DIPL's override mechanism so their assignments take effect before evaluation; do not assume that placing an ordinary file last has override semantics. Keep example-specific values in a short `settings.dip` or `profile.dip` and use additional files only for substantial tables or data. Explicitly map the Hub example ID to the pinned upstream input in `setups.json`, even if their names happen to match.

3.7 Discover and express meaningful dependencies

When transcribing upstream examples, compare their input files, documentation, and setup scripts to find values that are determined by other choices: unit conversions, dimensions, complementary fractions, bounds, or repeated references to the same physical quantity. Record which values are independent user choices and which are consequences. Do not copy a dependent value into every example merely because the native format asks for it separately.

Use a DIPL expression and an explicit node reference when the relationship is deterministic, stable for the supported code revision, and short enough for a contributor to review. Prefer one authoritative input and derive the other value from it. For example, if the solver needs both a box length and its midpoint, a DIPL expression can derive the midpoint from the length instead of repeating two literals:

```
geometry
  box_length float = 10.0
  box_midpoint float = ( {?geometry.box_length} / 2 )
```

Document the meaning and units in the actual schema. Test that changing the independent value through an override updates the dependent value and generated native input. Check computed values against pinned upstream examples. Do not impose a formula where the two upstream values can legitimately vary independently; explain that choice in the project guide.

DIPL is a parameter model, not a replacement simulation program. Leave iterative numerical work, data processing, file generation, and solver algorithms in the scientific code or a reviewed adapter. A long chain of branches or calculations that is hard to inspect is a sign to simplify the model or move the logic to an appropriate component. Do not invent derived nodes solely to make a dependency graph look connected. Avoid hidden copies and cycles, and explain non-obvious conversions near their definitions. Where a native file needs another unit or representation, make the transformation visible in the model or renderer documentation and test the result.

3.8 Keep provenance inspectable

The Hub's parameter reference shows evaluated paths, values, metadata, and source locations from a checked-in SNT report. Its graph distinguishes path hierarchy, schema supply, and **recorded evaluation reads**. A graph link is not automatically a scientific dependency; a literal setting may have no recorded reads. The report describes the bundled example, while a user's override can change effective values and dependencies. The generated DIPH5 snapshot and setup lock belong to each local run.

4 Adapter and workspace behavior

The project adapter converts the evaluated model to the solver's native inputs. It owns code-specific formatting, file names, ordering, and checks. The shared Hub runtime owns source-revision checks, safe staging, workspace discovery, setup locks, and common adapter arguments. The SNT3 command should infer the project from a local `.snthub/lock.json` after `snt hub fetch PROJECT`.

The usual local flow is:

```
mkdir my-study && cd my-study
snt hub fetch PROJECT
snt hub examples
snt hub setup EXAMPLE
snt hub build
snt hub run --setup runs/EXAMPLE
```

`build` and `run` are optional project capabilities. A setup-specific build needs `--setup runs/EXAMPLE`; an inputs-only recipe needs `--inputs-only` at setup and cannot be treated as ready to run. A per-run `--override-file` changes evaluated values without editing the fetched bundle. Project documentation must state compilers, external assets, parallel launch requirements, and any missing workflow steps.

5 Evidence and publication checklist

A project is ready for its declared registry state when its evidence matches its claims:

1. The source submodule, project record, setup manifest, and local examples agree on project ID and pinned revision.
2. Every published example parses with the supported SNT3 version. Public nodes have useful descriptions, units or unit conventions, constraints, native mappings where applicable, and source provenance. If an output plan is claimed, its recorded mappings cover the settings used to render native inputs and replay produces the same files under the documented asset requirements.
3. Generated native settings match pinned upstream examples under documented comparison rules. Explain intentional differences and preserve path-sensitive values.
4. A complete recipe stages all required inputs, including checked assets or initial data, from a clean workspace. Tests cover a failed setup without a published partial directory and safe override behavior.
5. Build and run claims have separate recorded checks, with machine, version, prerequisites, and observed result. Scientific-result validation needs its own reference and tolerance criteria.
6. `python3 scripts/test_all.py` passes, `python3 scripts/generate_project_docs.py --project <id> --snt <path-to-snt>` refreshes its parameter and schema exports, and the registry page links to the project guide, source, adapter, and parameter reference.

Independent adapters may be published with limited evidence when their scope and limitations are explicit. A project advances its support or validation label only after the corresponding review or check has occurred.

6 Related resources

- SNT3 source and SNT3 documentation cover the toolkit and current installation and CLI guidance.
- SNT initiative explains the broader collaboration and its goals.
- DIPL specification is authoritative for syntax and evaluation semantics; the adapter guide covers native-file generation.
- C++ DIPL guide, Python binding, and static parameter generation describe paths beyond an external adapter.
- Parameter Viewer documents live DIPL and DIPH5 inspection.
- SNT Hub source, project registry, integration levels and statuses, and new-project guide provide the current publishing context.

7 Maintaining this document

The Markdown source is `docs/INTEGRATION_SPEC.md`; `docs/pandoc.yaml` holds the Pandoc build settings, includes the colored Hub logo, and reads `VERSION` for the title. Compile the website PDF by hand from the repository root:

```
pandoc --defaults docs/pandoc.yaml
```

Pandoc and XeLaTeX are required. To write a copy elsewhere, add `-o integration-spec.pdf`. The helper script runs the same defaults file:

```
scripts/build_integration_spec.sh
```

The defaults file writes `website/public/integration-spec.pdf`. Astro copies that file to `/snt-hub/integration-spec.p` during the GitHub Pages build. Commit the Markdown, Pandoc assets, and PDF together so the downloadable edition matches the current specification.